

Para Empirismo:

Cada proy. es una instancia única.

nos define q' hay q' hacer.

Conjunto de acts. q' se adaptan al proyecto.

Esencia de un Proyecto Exitoso

Persona

Proceso

Proyecto

Producto

(PUD)
Definidos
Empíricos

La XP SI es extrapolable.

XP NO es extrapolable
se construye probando y equivocándose

Lean (Kanban)

Ajile → SCRUM

EMP. ASUME:

Autogestionadas:

capaces de comprometerse y cumplir. Responsables.

Motivadas

Capacitadas

* Por eso SCRUM no dice cómo hacer SFTW.

Es ÚNICO! : c/ vez q' hago SFTW obtengo un prod. único y q' aporta valor al cliente

CRISIS DEL SFTW

El término IS nace en 1968 (NATO)

x' la crisis del SFTW. Síntomas: el prod. no satisface al cliente, cuesta más tiempo y plata, tiene mala calidad y es difícil de mantener.

➤ NOS VA BIEN POR: involucrar al user, apoyo gerencial, req. claros.

➤ NOS VA MAL POR: req. incompletos, falta de involucrar al user, falta de recursos.

PROY. FALLIDOS

➤ Ariane 5 (conversión de 64 a 16 bits)

➤ Mars climate Orbiter (unidades vs. métricas).

➤ Toyota (SFTW de frenos)

SOFTWARE

→ Conoc./INFO. q' se nos presenta en ≠ modos de abstrac. Cualq. cosa q' haga q' entregue valor al proy. o entregar a mi cliente es SFTW.

ING DE SFTW

→ Disciplina q' trabaja desarrollando

saberes vinculados a las 4 dimensiones del desarrollo de SOFTWARE (PPPP)

DISCIPLINAS DE SOPORTE → Funcionan como paraguas transversal a todos los procesos, "dar soporte a todos"

→ SCM [sw config. management]

→ Aseguramiento de calidad de P/P [PPQA]

→ Métricas

SCM → Disciplina protectora transversal cuyo prop. es garantizar integridad del prod de SFTW a lo largo de toda la vida del producto

→ IC (Items de Config.) → C/ uno de los artefactos q' forman parte del prod o proyecto y cumplen: - Pueden sufrir cambios
- Nec. compartirse c/ el equipo - Nec. conoc. su estado evoluc.

→ Version vs. Variante → Version cambia en el tiempo (ej. IC v1, v2)
Variante coexiste en paralelo (ej. app p/ Windows/Linux/apple).

→ Línea Base → IC o conj. de ICs estables q' están validados y x' ende son confiables. Se puede desarrollar en base a esta Línea Base. Debe ser aprobada x' el Comité de Cambios.

→ Configurac de Software → "foto" q' se le saca al Repo q' contempla q' items tengo, en q' version y en det. momento.

→ Ramas → Bifurcan el desarrollo desde la rama principal y permiten experimentar.

→ Merge → Integrar una rama a la principal.

7 PROBLEMAS DEL MANEJO DE COMPO

- 1) Pérdida de un Compo
- 2) Pérdida de Cambios
- 3) Regresión de fallos
- 4) Falta de sincronía entre fuente, obj y ejecutable.
- 5) Doble mantenimiento
- 6) Superposición de Cambios
- 7) Cambios no validados

4 ACTS DEL SCM

- 1) Identificar IC: Def. q' artefactos son IC. Darles nombre y ubicarlos.
- 2) Control de Cambios: Proced. formal p/ cambiar IC en Línea Base
- 3) Auditorías: Verif. q' logi dice lo LB exte. q' el prod cumple lo req.
- 4) Informes de estado: Req. y report evolucion de IC

NO SILVER BULLET → (Brooks, 1987)

- No existe una "bala de plata", es decir, ninguna tecno. o técnica q' por sí sola mejore 10 veces la productividad, confiabilidad o simplicidad del SFTW.
- Esencia vs. Accidente:
 - Accidente → dif. de rep el SFTW: sintaxis, límites de memoria, tmp de compilación. (Ya casi resueltos).
 - Esencia → dif. del concepto: construir estruct. conceptuales complejas como algoritmos, datos y relaciones. Esto es lo difícil de verdad
- 4 props. esenciales del SFTW:

- 1) Complejidad: no hay 2 partes iguales y crece no lineal
- 2) Conformidad: Tiene q' adaptarse a sist. definidos x' otros.
- 3) Mutabilidad: Siempre bajo presión de cambio
- 4) Invisibilidad: No tiene representación geo o física nat.

Procesos Definidos → PUD

Proveen repetibilidad, y esa rep. me da visibilidad a la hora de hacer un proj. y definir tiempos, costos y prever errores

→ Asume q' podemos rep el mismo proc. una y otra vez y obtener los mismos result. algo q' el empirismo contradice, critica ya que plantea q' el producto es único.

Procesos Empíricos

→ Nos permiten aprender de la xp del equipo, la cual genera trabajando.

- Surgen del rechazo a los PD ante la nec. de ciclos de feedback RÁPIDO.
- Ej: Agile → Scrum → Trabajar en ciclos cortos p/ obtener realimentación del prod. y proc. Permite hacer cambios sin pasar x' alq. q' lo apruebe al ser un equipo autogestionado, motivado y capacitado. Se enfoca más en el hacer y no tanto en documentar. PERO SI DOCUMENTAN.

PILARES DEL ENTADISMO:

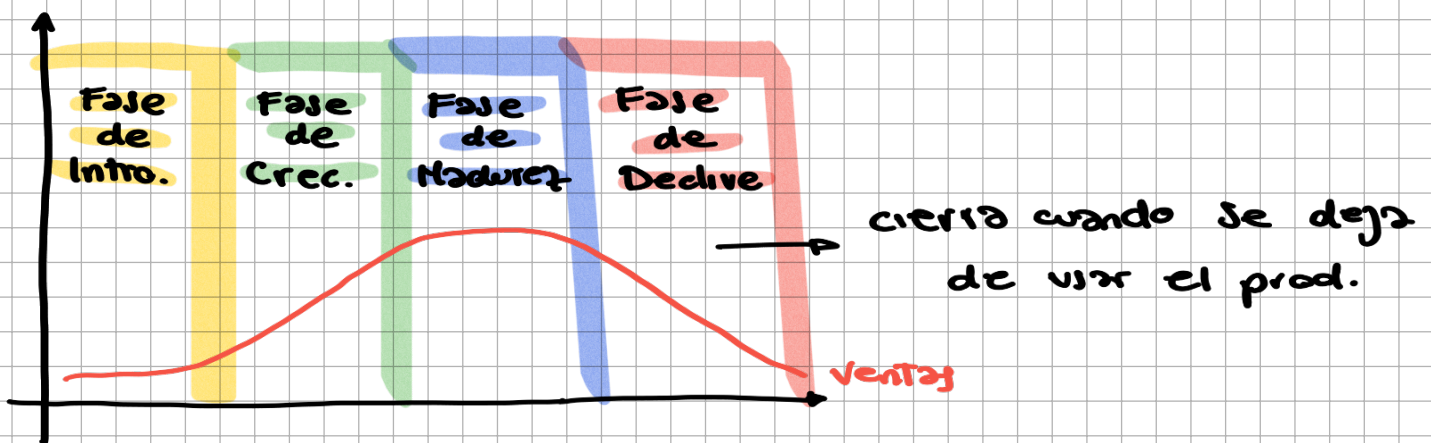
- > Inspección => Revisar c/ frec. los artefactos y progreso para detectar variaciones y problemas a tiempo
- > Adaptación => Adaptarse (cambiar) a errores y aprendizajes
- > Transparencia => No esconder errores ya q' de ellos se aprende y gana XP (principio del EMP).

Ciclo de Vida del Proyecto o Modelo de Proc.

- Secuencial (Cascada)
- Iterativa (I e Inc.)
- Recursivo (Espiral)

- Representa de un proceso q' especifica las fases y la que avanza el proyecto y el orden en q' se llaman a ca
- PROCESO DE DESARROLLO → define q' se hace, quien lo hace y cómo, con det., roles, artefactos y herramientas. El ciclo de vida solo define q' fases hay y en q' orden se ejecutan.

Ciclo de Vida del Producto



PROYECTO → unidad de Gestión de Trabajo. Medio x el cual se org. los recursos y por q' van a interactuar p/ obtener un prod. c/ valor agregado p/ el cliente.

CARACTS:

- Result. Único
- Elaboración Gradual
- Tiene Fecha Inicio y Fin
- Tareas interrelacionadas.

AGILISMO

Ideología c/ un conj. definido de ppios que guían el desarrollo de un prod

4 VALORES ÁGILES → Manifiesto Ágil

• Individuos e Iteraciones

SOBRE

Procesos y Herramientas

Software Funcionando

SOBRE

Documentación Exhaustiva

Colaborar c/ el cliente

SOBRE

Negociación Contractual

Responder al Cambio

SOBRE

Seguir un Plan

¿qué le enseñan

¿qué le enseñan

¿qué le enseñan

Entrega RAPIDA de software FUNCIONAL (incremental)

• 12 PRINCIPIOS DEL MANIFIESTO ÁGIL

- ★ Los responsables de negocios y desarrolladores deben trabajar juntos día a día durante el proy.
- ★ Las mejores org., requisitos y diseños emergen de equipos auto-organizados.
- ★ El método más eficiente de comunicar info. es en conversaciones cara a cara.

→ REQUERIMIENTOS ÁGILES

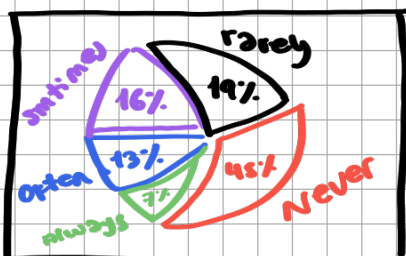
Vamos a partir de una vision de prod. q' la trae el PO y hacer un producto (NO PROTOTIPO) desplegable q' cumpla c/ SUS NECESIDADES es decir q' le de **VALOR**

→ "Usar el valor p/ construir el producto correcto"

→ OUTPUT: salida de código
≠
OUTCOME: salida q' le sirve al cliente

No conformarse c/ q' compile y ande, sino que de valor y el cliente quede contento.

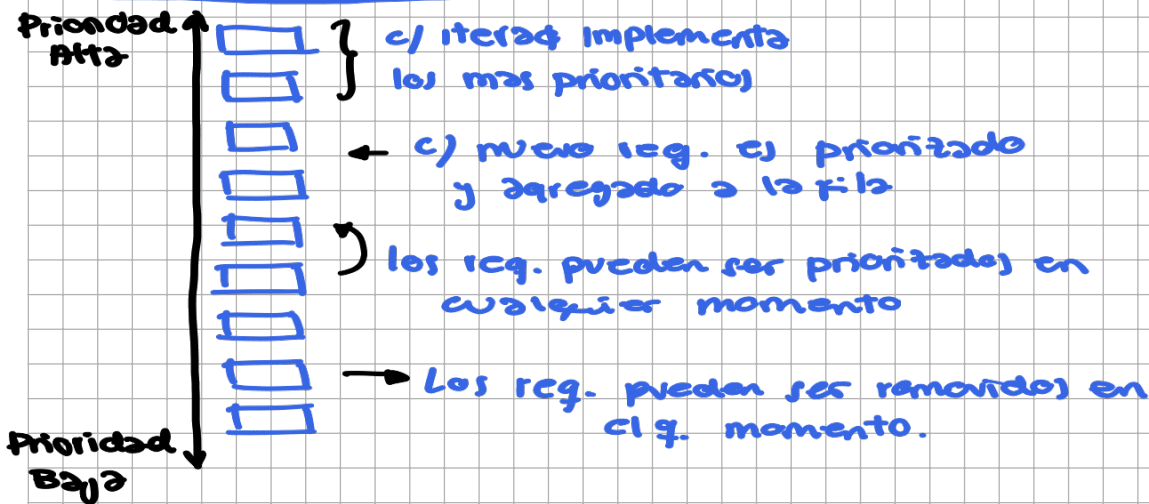
→ Los prod exitosos tmb. tienen un desperdicio significativo.



5 TIPOS DE REQUERIMIENTOS:

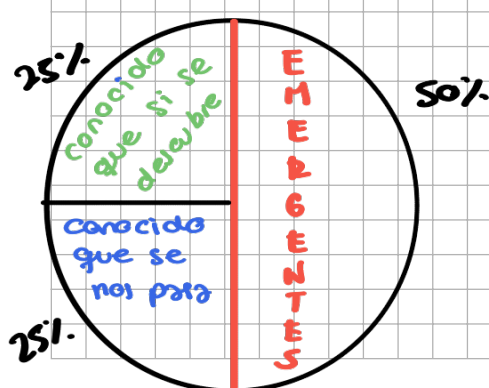
- > De negocio: el obj. de la org. Ej: Disminuir un % el t de cobro.
- > De usuario: lo q' el user nec. hacer. Ej: Consultar online estado...
- > Funcional: lo q' el sist. debe hacer. Ej: Generar reporte de saldo
- > No Funcional: Como debe hacerlo. Ej: Reporte en PDF
- > De implementaci3n: Restricciones t3cnicas de la soluci3n. Ej: cloud serv.

PRODUCT BACKLOG



REQ. EMERGENTES

El ágilismo acepta (y critica del PD) q' NO se puede escribir la totalidad de los req. en la ERS x' más q' sea el mejor equipo. Por eso dice q' tenemos q' ser capaces de trab. c/ una visi3n del prod q' voy a necesitar. Aceptando q' el 50% de req. que voy a implementar sean **EMERGENTES** (surgen mientras nos hacemos expertos en el prod) c/ el paso de las iteraciones. A su vez, el 25% de los req. son **CONOCIDOS** y el otro 25% se pagan por ALTO (errores de relevamiento).



Por eso el ágilismo dice q' comencemos el proy. sin todos los req. descubiertos



USER STORIES

Describe CORTA de una funcionalidad q' representa una
NECESIDAD PARA EL CLIENTE

3 partes → 3C → Cord → se ve
Conversation }
Confirmation } no se ve
parte + imp. }
Aquí el PO cuenta que necesita
como va a confirmar el PO q' hace lo q' se pidió (pruebas de aceptación)

Forma de Expresar:

Como <nombre del rol> QUE QUIERO <actividad>

Para Qué <valor q' recibe el negocio>
sirve p/ priorizar tmb.

→ PORCIONES VERTICALES:

Cada US atraviesa todas las capas (UI, log de neg y DB) y entrega funcionalidad completa y usable. No se corta x' capa técnica.

→ CRITERIOS DE ACEPTACIÓN:

- > Def. límites de la US > Permiten derivar las pruebas
- > Ayudan al PO a decir q' nec. p/ q' tienen valor > Indican cuando dejar de agregar funcionalidades.
- > Dan al equipo una visión compartida
- ¿Que los hace buenos? → Def. una intención, no una solución. Son indep. de la implementación y son de alto nivel.

→ PRUEBAS DE ACEPTACIÓN:

- Expresan los detalles q' surgieron de la conversación y complementan la US. "Probar [situación] [PASO]."

→ MODELO INVEST: Sirve p/ evaluar la calidad de una US.

- **I**ndependent (se puede implementar en clq. orden)
- **N**egotiable (Describen el QUE, el COMO se negocia con equipo).
- **V**aluable (debe tener valor p/ el cliente).
- **E**stimable (se debe poder estimar su esfuerzo, ayuda a priorizar).
- **S**mall (debe ser lo suficientemente peg. p/ hacerla en 1 iteración).
- **T**estable (se puede demostrar que fue implementada).

→ EPICA: una US muy grande q' hay q' dividir

TEMA: conjunto de varias US relacionadas

US: La funci3n descrita desde persp. de user, entra en 1 iteraci3n

→ SPIKES:

- > Tipo especial de US q' sirve p/ reducir riesgo o incertidumbre antes de comprometerse a desarrollar una funcionalidad.
- > T3cnicos → Invezt. enfoque t3cnicos (perfo., hacer/comprar)
- > Funcionales → resuelve dudas sobre interacci3n del user c/ el sist.
- > Lineamientos:
 - Tienen q' ser estimables, demostrables y aceptables.
 - Son la excepci3n, no la regla
 - Van en una iteraci3n ≠ a la US q' destraban, salvo q' sean muy chicos